



QRAPID

Operations Hub Architecture

Technical Architecture Document — P0

QRapid Technologies LLP | Internal Document | July 2026

Table of Contents

- [1. Purpose](#)
- [2. Stack](#)
- [3. Environments \(dev OFF by default\)](#)
- [4. Database Decision: PostgreSQL](#)
- [5. Data Model \(PostgreSQL via Prisma\)](#)
- [6. Razorpay Integration Boundary \(do not overbuild\)](#)
- [7. Communications \(MSG91\)](#)
- [8. Key Flows \(see the flow companion doc for the tree\)](#)
- [9. Security and Audit](#)
- [10. Repository and Environments](#)
- [11. Open Technical Questions](#)

Field	Value
Author	Vipul Katarnaware (senior dev review)
Status	v1, pairs with the P0 PRD
Scope	Production build on AWS
Last Updated	15 July 2026

1. Purpose

Technical contract derived from the P0 PRD. Defines stack, data model, the two-lifecycle design, Razorpay integration boundary, communications, environments, and key flows. The Lovable prototype validated flows; the production UI is built on the shadcn admin template (satnaing/shadcn-admin).

2. Stack

Layer	Choice	Notes
Frontend	React + TypeScript + Vite + Tailwind, shadcn/ui via the satnaing/shadcn-admin template	Template is the component and layout base. Hosted on AWS Amplify Hosting or S3 + CloudFront.
Backend	NestJS (Node.js + TypeScript)	Structured modules, DI, validation, testing.
Database	PostgreSQL on AWS RDS	Relational domain (state machines, payments, audit). See Section 4.
ORM	Prisma	Type-safe schema shared with the frontend; migrations built in.
Auth	NestJS + Passport JWT (access + refresh), bcrypt	One role in P0; revisit for RBAC in P1.
File storage	AWS S3	Menu files, QR sheets, generated assets. Private bucket, pre-signed URLs.
Payments	Razorpay Subscriptions + Payment Links + Webhooks	Razorpay owns billing lifecycle, reminders, retries. See Section 6.
Messaging	MSG91 (SMS + WhatsApp)	Payment/renewal links, first-login OTP, handover link.
AI extraction	OpenRouter (low-cost vision model)	Runs in the restaurant dashboard config path, not the Ops Hub.
Compute	AWS App Runner (NestJS container)	Managed HTTPS, autoscaling. ECS Fargate fallback.

Layer	Choice	Notes
Secrets	AWS Secrets Manager	Razorpay keys, MSG91 keys, DB creds, JWT secrets.
Monitoring	CloudWatch + Sentry	Alarms on webhook failures.
CI/CD	GitHub Actions -> App Runner + Amplify	Deploy on merge to main.

Estimated monthly cost at current scale: RDS db.t4g.micro ~ \$15-25, App Runner ~ \$25-50, S3/CloudFront under \$5. Roughly \$50-80/month, so the \$1000 AWS credits cover 12-18 months, and the same setup comfortably serves 2,000-5,000 restaurants before any resize.

3. Environments (dev OFF by default)

- Two environments: production (always on) and dev/staging (OFF by default).
- The dev/staging environment is powered on ONLY during active development or staging tests, then shut down, to control cost and reduce exposure. Implement as an App Runner service + RDS instance that are started on demand (or an RDS instance that is stopped when idle) and a documented start/stop runbook or a scheduled auto-stop.
 - Local development uses Docker Postgres so day-to-day work does not require the cloud dev env.
 - Secrets are per-environment; no shared credentials between prod and dev.

4. Database Decision: PostgreSQL

MongoDB Atlas credits were available and considered. PostgreSQL on RDS is chosen because the domain is relational: restaurants link to owners, rooms, tables, subscriptions, payments, transitions, menu items, QR codes. Foreign keys and constraints (one non-archived subscription per restaurant, unique restaurant codes, unique Razorpay payment IDs) are enforced by the DB rather than application code. The state machine needs transactional integrity (a payment webhook atomically records payment, transitions state, writes audit and timeline). Money and audit favor strict schema, and P1 reporting is plain SQL. The Atlas credits save ~\$15-25/month, which the AWS credits already cover on RDS; credits should not drive a foundational data-model choice. Atlas can serve a document-shaped secondary store later if needed (e.g. raw AI extraction payloads).

5. Data Model (PostgreSQL via Prisma)

Two independent state fields per restaurant: setup_state and (on subscriptions) status.

restaurants

- id (uuid pk), restaurant_code (unique immutable; QR-YYYY-NNNN, R-2)
- name, type, description, pincode, state, city, area, address1, address2
- opening_time, closing_time, gst_enabled, gst_number, fssai_number

- source (inbound|outbound), timezone default Asia/Kolkata (hidden)
- setup_state (draft|pending|completed)
- created_at, created_by

owners

- id, restaurant_id (fk unique), name, primary_whatsapp (not null), primary_email (as typed)
- additional_mobilis (jsonb), additional_emails (jsonb)

rooms / tables

- rooms: id, restaurant_id, name, table_count, is_parcel, sort_order
- tables: id, room_id, table_number, qr_code_id (nullable). numbering_mode on restaurants.

plans

- id, name, base_price (paisa), billing_cycle (annual), razorpay_plan_id, active

subscriptions

- id, restaurant_id (fk), plan_id (fk), base_price_snapshot, flat_discount, percent_discount, final_price (paisa)
- status (trial|active|renewal_grace|payment_due|suspended|archived)
- status_reason (trial|renewal, when payment_due)
- billing_mode (online|cash|complimentary)
- razorpay_subscription_id, trial_ends_at, renewal_date, current_status_ends_at, suspended_at, archived_at
- partial unique index: one non-archived subscription per restaurant

status_transitions (append-only)

- id, subscription_id, from_status, to_status, reason (razorpay_webhook|manual|migration), actor, created_at

payments

- id, subscription_id, amount (paisa), method (online|cash|complimentary)
- razorpay_payment_id (unique nullable), razorpay_link_id (nullable)
- status (created|paid|failed), paid_at, recorded_by

menu_uploads / menu_items (restaurant-dashboard config; shared DB)

- menu_uploads: id, restaurant_id, storage_key, extraction_status, model_used, cost_paise, error_detail
- menu_items: id, restaurant_id, name, price (paisa), category, is_veg, status (draft|published)

tax_config

- id, restaurant_id (fk unique), rate (5|18|composition), pricing_mode (inclusive|exclusive)
- service_charge_enabled, service_charge_percent. Optional; not required for handover.

qr_codes

- id, code (unique), restaurant_id (nullable), table_id (nullable), assigned_at, assigned_by

admin_users

- id, name, email (unique), password_hash, role (super_admin), last_login_at

setup_credentials

- id, restaurant_id (fk unique), username_mobile, temp_password_hash, generated_at, owner_first_login_at

timeline_events

- id, restaurant_id, kind (billing|operations|notes), title, detail (jsonb), actor, created_at

internal_notes

- id, restaurant_id, body, author, created_at (admin-only, RLS)

Money as integer paisa. Timestamps timestamptz; business-day logic in Asia/Kolkata.

setup_state derivation: pending -> completed when a restaurant has at least one published menu_item AND at least one table. This is computed on write (menu approve / table save) and persisted, so the Ops Hub reads a stored flag rather than recomputing.

6. Razorpay Integration Boundary (do not overbuild)

Division of labor, per the resolved clock-ownership decision (PRD Section 11, Option A): QRapid owns the lifecycle CLOCK; Razorpay is the payment RAIL.

Razorpay owns:

- Plans mapping (razorpay_plan_id), hosted payment pages, and payment links for trial conversion and renewal.
- Payment collection and retries, and the built-in Payment Link reminders (up to 3, SMS and email only; Razorpay sends no WhatsApp).
- Webhooks as the source of truth for PAYMENT events (payment captured/failed). The webhook handler verifies the signature, upserts the payment by razorpay_payment_id (idempotent; replays are no-ops), and calls the single transition service, which writes status_transitions + timeline_events in one transaction.

QRapid owns:

- The lifecycle clock: every alive stage's deadline (surfaced as Payment Due Date), the daily aging job (EventBridge -> /jobs/lifecycle-daily, advisory-locked, idempotent) that advances elapsed stages, and synchronous transitions when an override sets a past date (PRD FR-45).
- All WhatsApp messaging via MSG91 (Razorpay cannot send WhatsApp), triggered off webhooks and lifecycle events.
- Manual overrides, offline payments, suspend/reactivate/restore, and the 90-day suspended-to-archived aging.

No auto-debit mandate in P0 (annual amount too large for autopay). Each renewal is a fresh hosted payment; the restaurant app is hard-blocked behind the Razorpay screen until payment succeeds, after which the webhook flips the subscription to active and recomputes renewal_date.

7. Communications (MSG91)

- MSG91 sends both SMS and WhatsApp.
 - Payment/renewal links: WhatsApp + SMS. Razorpay Payment Link reminders supplement.
 - Credentials: shown once in-dashboard (copy), never SMS'd as a password.
 - Owner first-login: SMS OTP.
 - Handover: WhatsApp login link when setup auto-completes.
 - All sends are triggered from NestJS via MSG91 APIs and logged to timeline_events.
-

8. Key Flows (see the flow companion doc for the tree)

- Onboarding: thin wizard -> atomic create (restaurant + owner + subscription + credentials + timeline) -> setup=pending. Activate Immediately creates a Razorpay link; trial starts a trial.
- Configuration/handover: Ops logs into the restaurant dashboard, adds menu + tables; on both existing, setup auto-completes and MSG91 sends the owner handover link.
- QR: global tool; camera scans a physical QR; bind writes the table's ordering link to that QR.
- Payments: Razorpay webhooks drive status; offline payments recorded manually with the same

transition path; every payment on the Timeline.

- Migration: a one-off NestJS command imports the existing base through the same services.

9. Security and Audit

- JWT on all endpoints except /webhooks/razorpay (signature-verified) and any job endpoint (secret header). Passwords and temp passwords hashed; temp password shown once.
- Secrets in AWS Secrets Manager; none in the repo.
- Append-only audit via status_transitions and timeline_events; no destructive deletes (archive model, reversible).
- S3 private with short-lived pre-signed URLs. RDS backups + 7-day PITR.
- P1: audited "Login as restaurant" impersonation replaces shared setup credentials after handover.

10. Repository and Environments

- Monorepo: apps/web (React on shadcn-admin), apps/api (NestJS), packages/shared (Prisma-generated types + the transition table constant shared by both sides).
- Environments: local (Docker Postgres), prod (always on), dev/staging (off by default, started on demand for testing).
- GitHub Actions: lint + tests on PR; deploy on merge to main. Seed script creates plans, an admin user, and a demo restaurant.

11. Open Technical Questions

1. Restaurant code format: confirm QR-YYYY-NNNN and yearly reset (PRD R-2).
2. Restore-from-archive pricing: snapshot vs current plan (PRD R-3).
3. OpenRouter model + accuracy benchmark on 10 real menus (PRD R-1).
4. Pincode data source: free API vs bundled dataset.
5. Exact mapping of Razorpay subscription states/webhooks to QRapid's mirrored statuses, especially renewal_grace vs payment_due timing, to be confirmed during M2 against a live Razorpay test account.

Note for maintainers: first version, aligned with PRD v1, not yet circulated. Single current version.